TRƯỜNG ĐẠI HỌC NHA TRANG

KHOA CÔNG NGHỆ THÔNG TIN

ĐỀ CƯƠNG ĐỀ TÀI

TÊN ĐỀ TÀI:

XÂY DỰNG HỆ THỐNG CÁNH TAY ROBOT JETMAX ĐÁNH CỜ TƯỚNG TỰ ĐỘNG

I. MỤC ĐÍCH

Nhằm giúp sinh viên vận dụng các kiến thức đã học về học sâu (Deep Learning), xử lý ảnh, thị giác máy tính (Computer Vision), trí tuệ nhân tạo trong trò chơi và điều khiển robot để giải quyết một bài toán thực tế: robot tự động quan sát bàn cờ, nhận dạng các quân cờ, tính toán nước đi và trực tiếp thi đấu cờ tướng với người chơi.

Đề tài xây dựng hệ thống sử dụng cánh tay robot Hiwonder JetMax (bộ điều khiển Jetson Nano, camera gắn trên robot, có thanh trượt mở rộng vùng hoạt động), có khả năng nhận dạng bàn cờ và đủ 14 loại quân cờ tướng (7 loại quân × 2 màu đỏ/đen) bằng mô hình học sâu YOLO11 kết hợp thư viện OpenCV, dựng lại thế cờ từ ảnh, dùng engine cờ tướng mã nguồn mở Pikafish [5] để chọn nước đi, sau đó điều khiển robot gắp quân đặt vào đúng ô trên bàn cờ (nếu ăn quân thì đưa quân bị ăn ra khay). Người chơi cầm quân đỏ thì đi trước và robot cầm quân đen đi sau; ngược lại, người chơi cầm quân đen thì robot đi trước. Khi camera không quan sát rõ toàn bộ bàn cờ từ một vị trí, robot di chuyển camera qua nhiều vị trí quan sát và gộp các kết quả nhận dạng thành một bàn cờ ảo thống nhất theo toạ độ bàn cờ.

II. NỘI DUNG

- Tổng quan về bài toán cờ tướng tự động: luật cờ tướng, cách biểu diễn thế cờ (lưới 9×10 giao điểm, ký hiệu FEN) và giao thức UCI; engine cờ tướng mã nguồn mở Pikafish [5]; sơ đồ tổng thể của hệ thống gồm nhận dạng bàn cờ và quân cờ, dựng thế cờ, chọn nước đi, điều khiển robot gắp - đặt.

- Tổng quan về bài toán phát hiện và phân loại vật thể bằng thị giác máy tính và học sâu; khảo sát, so sánh các hướng tiếp cận phát hiện đối tượng (xử lý ảnh truyền thống, hai giai đoạn, một giai đoạn, Transformer, họ YOLO) và lý do lựa chọn YOLO, đặc biệt là YOLO11 [1]–[3], [9]–[21]; các nghiên cứu liên quan về nhận dạng quân cờ tướng [4].
- Tổng quan về hệ thống robot gắp - đặt: phân tích bài toán, ràng buộc phần cứng (tầm với, thanh trượt, đầu hút) và lựa chọn robot JetMax [6] cùng camera gắn trên robot; hiệu chuẩn bàn cờ và khay quân bị ăn; xây dựng chiến lược quan sát từ nhiều vị trí và gộp kết quả thành bàn cờ ảo (biến đổi phối cảnh từ ảnh sang lưới 9×10, rồi sang toạ độ robot); xác định 14 lớp quân cờ cần phân loại.
- Tổng quan về dữ liệu huấn luyện cho mô hình phát hiện đối tượng; lựa chọn và lý do sử dụng các bộ dữ liệu quân cờ tướng công khai trên Roboflow Universe [8]; cách hợp nhất, chuẩn hóa về 14 lớp và chia tập dữ liệu.
- Huấn luyện mô hình YOLO11, kết hợp OpenCV để kiểm tra chéo kết quả, xác định vị trí quân cờ trên bàn và chuyển đổi tọa độ; triển khai mô hình lên thiết bị biên (xuất ONNX/TensorRT).
- Xây dựng phần mềm điều khiển ván cờ: nhận ra nước đi của người chơi bằng cách so sánh thế cờ trước - sau và đối chiếu luật cờ tướng; gọi engine Pikafish qua giao thức UCI; điều khiển robot gắp - đặt (kể cả nước ăn quân); xử lý các tình huống bất thường (nước đi sai luật, nhận dạng nhiễu, lỗi cơ khí); mô phỏng toàn bộ hệ thống trước khi chạy trên robot thật.
- Thực nghiệm trên robot thật: đánh giá độ chính xác nhận dạng quân cờ, tỉ lệ nhận đúng nước đi của người chơi, tỉ lệ gắp - đặt thành công, thời gian cho mỗi nước đi, độ lặp lại của các vị trí quan sát; cho robot thi đấu hoàn chỉnh nhiều ván ở cả hai phe và các mức độ của engine; từ đó cải thiện hệ thống.

a) Khảo sát các hướng tiếp cận phát hiện đối tượng và lý do chọn YOLO11

Bài toán nhận dạng ở đây là phát hiện đồng thời vị trí và loại của tối đa 32 quân cờ (14 lớp) trong mỗi ảnh, chạy trên thiết bị biên Jetson Nano. Mỗi lượt đi robot có thể phải quan sát bàn cờ ở nhiều vị trí nên thời gian suy luận cho mỗi ảnh phải thấp. Đề tài khảo sát năm nhóm phương pháp, tóm tắt ở Bảng 1.

Bảng 1. So sánh các hướng tiếp cận phát hiện đối tượng đối với bài toán nhận dạng quân cờ tướng

Nhóm phương pháp (đại diện)	Ưu điểm	Hạn chế đối với đề tài
Xử lý ảnh truyền thống: biến đổi Hough tìm đường tròn [18], HOG + SVM [19], đối sánh mẫu	Nhẹ, không cần tập dữ liệu huấn luyện lớn, chạy được trên CPU.	Nhạy với ánh sáng, bóng đổ và phản chiếu trên mặt quân; chữ Hán khắc nhiều nét khó phân biệt bằng đặc trưng thiết kế thủ công; phải chỉnh tham số theo từng điều kiện; kém ổn định khi quân bị che một phần.
Hai giai đoạn: Faster R-CNN [9]	Độ chính xác cao, định vị tốt.	Phải đề xuất vùng rồi mới phân loại nên chậm và nặng hơn; khó đáp ứng thời gian thực trên Jetson Nano, độ trễ cộng dồn khi mỗi lượt quét nhiều vị trí.
Một giai đoạn dựa trên anchor: SSD [10], RetinaNet [11], EfficientDet [12]	Nhanh hơn mô hình hai giai đoạn; một lần suy luận cho toàn ảnh.	Cần thiết lập và tinh chỉnh anchor; ít công cụ tích hợp liền mạch cho huấn luyện - đánh giá - xuất mô hình sang ONNX/TensorRT so với hệ sinh thái Ultralytics.
Dựa trên Transformer: DETR [13], RT-DETR [14]	Không cần bước loại trùng hộp (NMS); RT-DETR đạt tốc độ thời gian thực trên GPU rời.	DETR hội tụ chậm, cần nhiều tài nguyên huấn luyện; mô hình nặng, yêu cầu bộ nhớ và tính toán lớn, ít phù hợp thiết bị biên cấu hình thấp như Jetson Nano.
Họ YOLO một giai đoạn không anchor: YOLOv8, YOLO11 [1]–[3], [15]	Một lần suy luận cho cả ảnh; cân bằng tốc độ và độ chính xác; có nhiều cỡ mô hình (n, s, m, l, x); huấn luyện, đánh giá, xuất ONNX/TensorRT bằng cùng một công cụ; nhãn YOLO là định dạng phổ biến.	Độ chính xác phụ thuộc chất lượng và độ đa dạng dữ liệu; có thể nhầm quân khi bị che hoặc ánh sáng kém. Giảm thiểu bằng cách bỏ phiếu qua nhiều lần quét và kiểm tra luật cờ tướng.

Lý do chọn họ YOLO [1]. (1) Là mô hình một giai đoạn, chỉ cần một lần suy luận để thu được vị trí và loại của mọi quân cờ trong ảnh, phù hợp yêu cầu thời gian thực trên thiết bị biên. (2) Trong ảnh chụp từ camera gắn trên robot, quân cờ là vật thể có kích thước vừa phải, hình dạng và kích thước tương đối đồng đều, nổi bật trên nền bàn cờ, nên không cần kiến trúc phức tạp hơn (hai giai đoạn hoặc Transformer) để đạt độ chính xác cần thiết. (3) Hệ sinh thái Ultralytics cung cấp trọn quy trình huấn luyện, đánh giá và xuất mô hình (ONNX, TensorRT) bằng cùng một công cụ. (4) Các bộ dữ liệu cờ tướng trên Roboflow Universe có thể tải trực tiếp ở định dạng nhãn YOLO.

Lý do chọn YOLO11 [2], [3]. (1) So với YOLOv8, YOLO11 thay khối C2f bằng C3k2 và bổ sung khối chú ý không gian C2PSA; theo công bố của Ultralytics, YOLO11m dùng ít hơn khoảng 22% số tham số so với YOLOv8m nhưng đạt mAP cao

hơn 1,3% trên tập COCO [20], tức mô hình nhẹ hơn mà không giảm độ chính xác, có lợi cho Jetson Nano. (2) YOLO11 có nhiều cỡ mô hình (n, s, m, l, x), cho phép chọn cỡ cân bằng giữa độ chính xác và tốc độ. (3) YOLOv12 [16] dựa trên cơ chế chú ý, nhưng Ultralytics ghi nhận có thể không ổn định khi huấn luyện, tốn bộ nhớ và chạy chậm hơn trên CPU, đồng thời khuyến nghị dùng YOLO11 hoặc YOLO26 cho triển khai thực tế [21]. (4) YOLO26 [17] là phiên bản mới nhất của Ultralytics (ra mắt tháng 01/2026), thiết kế không cần NMS và ưu tiên thiết bị biên; tuy nhiên phiên bản này còn mới và Ultralytics cho biết chưa công bố bài báo chính thức cho YOLO26 [17], trong khi YOLO11 đã được mô tả đầy đủ trong tài liệu học thuật [3], được cộng đồng sử dụng rộng rãi và đã được huấn luyện trên bộ dữ liệu của đề tài. Việc so sánh thực nghiệm YOLO11 với YOLO26 trên cùng dữ liệu và cùng phần cứng được xem là hướng mở rộng.

Kết quả của mô hình được kiểm tra chéo bằng OpenCV và bằng luật cờ tướng (thế cờ nhận dạng phải hợp lệ) để giảm ảnh hưởng của lỗi nhận dạng.

b) Dữ liệu huấn luyện: nguồn, cách sử dụng và lý do lựa chọn

Đề tài sử dụng các bộ dữ liệu công khai thay vì tự chụp toàn bộ, vì thu thập và gán nhãn hàng nghìn ảnh cho 14 lớp tốn nhiều thời gian; ảnh chụp từ camera robot được bổ sung khi cần. Bốn bộ dữ liệu được chọn từ Roboflow Universe [8] được nêu ở Bảng 2.

Bảng 2. Các bộ dữ liệu quân cờ tướng được sử dụng

Bộ dữ liệu	Nguồn (Roboflow Universe)	Quy mô / phiên bản
Xiangqi-Super-Dataset	312s-workspace/xiangqi-super-dataset	Khoảng 4.460 ảnh
Xiangqi-Scara-YOLO26	312s-workspace/xiangqi-scara-yolo26	892 ảnh
new-chinese-chess	quoc-rnmzr/new-chinese-chess-ayph4	Phiên bản 1
XiangQi	duytranle/xiangqi-ubjij	Phiên bản 18
Tập sau khi hợp nhất và chuẩn hóa nhãn	**Hợp nhất từ bốn bộ trên**	**6.808 ảnh, 14 lớp**

Cách sử dụng dữ liệu:

- Tải các bộ dữ liệu bằng mã lệnh (Roboflow) trên Google Colab ở định dạng nhãn YOLO, ghi rõ nguồn và phiên bản để có thể tái lập.
- Kiểm tra chất lượng nhãn; ánh xạ tên lớp của từng bộ về cùng 14 lớp chuẩn (7 loại quân × 2 màu); loại các lớp hoặc ảnh không thuộc 14 lớp; loại bỏ ảnh trùng lặp hoặc gán nhãn sai.

- Hợp nhất thành một tập 14 lớp gồm 6.808 ảnh, kiểm tra mức độ cân bằng giữa các lớp.
- Chia tập huấn luyện/kiểm định/kiểm thử theo tỉ lệ khoảng 80/10/10 và chia theo nhóm ảnh gốc, để các phiên bản biến đổi của cùng một ảnh không rơi vào nhiều tập, tránh rò rỉ dữ liệu làm mAP cao giả tạo.
- Huấn luyện YOLO11 bằng học chuyển tiếp (transfer learning) trên Google Colab; đánh giá mAP@0.5 trên tập kiểm thử.
- Đánh giá lại trên ảnh chụp từ camera robot (bàn cờ, bộ quân và ánh sáng thực tế); nếu độ chính xác giảm đáng kể thì bổ sung ảnh tự chụp, tự gán nhãn và tinh chỉnh (fine-tune) mô hình.

Lý do lựa chọn các bộ dữ liệu này:

- Là dữ liệu công khai, đã gán nhãn hộp giới hạn (bounding box) ở định dạng YOLO, đúng với bài toán phát hiện đối tượng và tiết kiệm công gán nhãn thủ công.
- Có nhãn theo từng loại quân và màu quân (hoặc ánh xạ được về 14 lớp), đáp ứng yêu cầu phân biệt đủ 14 lớp. Các bộ chỉ gán 2 lớp đỏ/đen (ví dụ chinese-chess-nucht) bị loại vì không đủ thông tin để phân biệt loại quân.
- Kết hợp nhiều nguồn cho phép đa dạng về bộ cờ, bàn cờ, ánh sáng và góc chụp, giảm nguy cơ mô hình chỉ học đặc điểm của một bộ cờ (quá khớp) và tăng khả năng khái quát hóa.
- Xiangqi-Super-Dataset có quy mô lớn nhất (khoảng 4.460 ảnh) nên là nền tảng của tập huấn luyện; các bộ còn lại bổ sung thêm sự đa dạng.
- Tổng số 6.808 ảnh sau hợp nhất phù hợp để tinh chỉnh mô hình YOLO11 cho 14 lớp bằng học chuyển tiếp.

Hạn chế và cách xử lý: dữ liệu công khai được chụp bằng nhiều loại camera, bàn cờ và bộ quân khác với hệ thống thực tế (khác biệt miền dữ liệu), nên kết quả trên tập kiểm thử có thể cao hơn khi chạy trên robot. Vì vậy đề tài đánh giá thêm trên ảnh từ camera robot và bổ sung dữ liệu tự chụp khi cần, như nêu trên.

III. YÊU CẦU

- Có kiến thức về các thuật toán phát hiện đối tượng (đặc biệt YOLO), xử lý ảnh với OpenCV, động học robot, lập trình ROS/SDK của JetMax [7] và giao thức UCI để giao tiếp với engine cờ tướng.

- Nguồn dữ liệu: sử dụng bốn bộ dữ liệu công khai trên Roboflow Universe (tìm theo từ khóa “Xiangqi” hoặc “Chinese Chess”) gồm Xiangqi-Super-Dataset và Xiangqi-Scara-YOLO26 (312s Workspace), new-chinese-chess (quoc-rnmzr) và XiangQi (DuyTranLe), đã gán nhãn sẵn ở định dạng YOLO [8]; lý do lựa chọn nêu ở mục b.
- Xử lý dữ liệu: kiểm tra chất lượng nhãn; hợp nhất các bộ dữ liệu và chuẩn hóa nhãn về đủ 14 lớp (7 loại quân × 2 màu), tương đối cân bằng giữa các lớp; loại bỏ ảnh trùng lặp hoặc gán nhãn sai; chia lại tập huấn luyện/kiểm định/kiểm thử (khoảng 80/10/10) theo nhóm ảnh gốc, tránh trùng ảnh giữa các tập; mô tả rõ nguồn dữ liệu và quy trình xử lý.
- Hệ thống nhận dạng bàn cờ và phân loại đủ 14 loại quân cờ, dựng được thế cờ chính xác; robot chơi được cả hai phe (đi sau khi người cầm quân đỏ, đi trước khi người cầm quân đen), chọn nước đi bằng Pikafish và gắp - đặt đúng ô, kể cả nước ăn quân; hoạt động được khi camera không quan sát toàn bộ bàn cờ từ một vị trí.
- Mục tiêu dự kiến (điều chỉnh sau khảo sát thực nghiệm): mAP@0.5 ≥ 95% trên tập kiểm thử; tỉ lệ nhận đúng thế cờ và nước đi của người chơi ≥ 90%; tỉ lệ gắp - đặt đúng ô ≥ 90%.
- Thực hiện demo thi đấu hoàn chỉnh giữa robot và người ở cả hai phe, đánh giá hiệu quả của mô hình và của hệ thống.
- *Ngôn ngữ và thư viện cài đặt:* Python; PyTorch, Ultralytics, OpenCV, NumPy; ROS và SDK của JetMax; ONNX, TensorRT; engine Pikafish (giao thức UCI).

TÀI LIỆU THAM KHẢO

[1] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. In Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 779–788.

[2] Jocher, G., & Qiu, J. (2024). Ultralytics YOLO11 (phiên bản 8.3.x) [Phần mềm]. GitHub. https://github.com/ultralytics/ultralytics

[3] Khanam, R., & Hussain, M. (2024). YOLOv11: An overview of the key architectural enhancements. arXiv preprint arXiv:2410.17725.

[4] Wijaya, R. S., Anadia, A. Y., Fatekha, R. A., & Prayoga, S. (2025). Real-time Chinese chess piece character recognition using Edge AI. Journal of Applied Informatics and Computing (JAIC), 9(4), 1984–1990.

[5] Pikafish Developers. Pikafish – A free and strong UCI xiangqi engine [Phần mềm]. GitHub. https://github.com/official-pikafish/Pikafish (truy cập 28/10/2026).

[6] Hiwonder Technology. JetMax – Tài liệu hướng dẫn (Tutorial) và mã nguồn [Tài liệu trực tuyến]. https://docs.hiwonder.com

[7] Quigley, M., Conley, K., Gerkey, B., et al. (2009). ROS: An open-source Robot Operating System. In ICRA Workshop on Open Source Software, vol. 3, no. 3.2.

[8] Roboflow Universe. Các bộ dữ liệu Xiangqi-Super-Dataset và Xiangqi-Scara-YOLO26 (312s Workspace), new-chinese-chess (quoc-rnmzr), XiangQi (DuyTranLe) [Bộ dữ liệu trực tuyến]. https://universe.roboflow.com (truy cập 28/10/2026).

[9] Ren, S., He, K., Girshick, R., & Sun, J. (2017). Faster R-CNN: Towards real-time object detection with region proposal networks. IEEE Transactions on Pattern Analysis and Machine Intelligence, 39(6), 1137–1149.

[10] Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.-Y., & Berg, A. C. (2016). SSD: Single shot multibox detector. In Proc. European Conference on Computer Vision (ECCV), pp. 21–37.

[11] Lin, T.-Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). Focal loss for dense object detection. In Proc. IEEE International Conference on Computer Vision (ICCV), pp. 2980–2988.

[12] Tan, M., Pang, R., & Le, Q. V. (2020). EfficientDet: Scalable and efficient object detection. In Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 10781–10790.

[13] Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., & Zagoruyko, S. (2020). End-to-end object detection with transformers. In Proc. European Conference on Computer Vision (ECCV), pp. 213–229.

[14] Zhao, Y., Lv, W., Xu, S., Wei, J., Wang, G., Dang, Q., Liu, Y., & Chen, J. (2024). DETRs beat YOLOs on real-time object detection. In Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 16965–16974.

[15] Terven, J., Córdova-Esparza, D.-M., & Romero-González, J.-A. (2023). A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS. Machine Learning and Knowledge Extraction, 5(4), 1680–1716.

[16] Tian, Y., Ye, Q., & Doermann, D. (2025). YOLOv12: Attention-centric real-time object detectors. arXiv preprint arXiv:2502.12524.

[17] Ultralytics. Ultralytics YOLO26 [Tài liệu trực tuyến]. https://docs.ultralytics.com/models/yolo26/ (truy cập 28/10/2026).

[18] Duda, R. O., & Hart, P. E. (1972). Use of the Hough transformation to detect lines and curves in pictures. Communications of the ACM, 15(1), 11–15.

[19] Dalal, N., & Triggs, B. (2005). Histograms of oriented gradients for human detection. In Proc. IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR), vol. 1, pp. 886–893.

[20] Ultralytics. YOLO11 vs YOLOv8: A comprehensive technical comparison [Tài liệu trực tuyến]. https://docs.ultralytics.com/compare/yolo11-vs-yolov8 (truy cập 28/10/2026).

[21] Ultralytics. Model comparisons và YOLO12 [Tài liệu trực tuyến]. https://docs.ultralytics.com/compare ; https://docs.ultralytics.com/models/yolo12/ (truy cập 28/10/2026).

SV - Email: trong.nh.65cntt@ntu.edu.vn Phone: 0935 918 965